phase5_rotation.sh — Let's Encrypt Phase-5 renewal/ rotation proof

Revision: 1 **Last modified:** 2026-07-01T11:40:00Z **Authority:** Inherits `constitution/Constitution.md` per §11.4.35. §11.4.18 companion for `deploy/letsencrypt/phase5_rotation.sh`.

Overview

Proves — repeatably, with captured evidence — that Caddy **renews** its certificate via the ACME renewal path (a genuine rotation to a NEW serial) and that the renewal **swap** (old leaf → new leaf) is **zero-downtime**. Builds on the proven Phase-3 issuance (`phase3_hermetic_issue.sh`), then forces exactly one renewal (§11.4.98 re-runnable · §11.4.107 real-evidence).

Why the storage-surgery trigger

certmagic v0.21.3 renews when a cert's **cached** ARI `_selectedTime` is in the past, but caches ARI persistently (cert `issuer_data`, `Retry-After` 6h) and re-fetches only when that elapses — neither `POST /load` nor a Caddy ≥2.11.0 bump forces a re-fetch (source-proven, `docs/research/caddy_2110_ari_refetch_20260701/`). The deterministic hermetic trigger is therefore: rewrite the cert's cached ARI window (`issuer_data.renewal_info.suggestedWindow` + `_selectedTime`) to the **past** in storage, then **restart** Caddy so it re-loads that past window → the next maintenance tick renews. **In production this trigger is unnecessary** — Caddy renews on its own schedule when the cert genuinely nears expiry (zero-downtime, no restart). The restart here is TEST SCAFFOLDING; the RENEWAL SWAP it induces is what this test measures for zero-downtime (availability is probed AFTER the restart, across the swap).

Prerequisites

- The built image `localhost/helix_proxy/caddy-challtestsrv:2.8.4` (run `build.sh`).
- Rootless `podman`/`podman-compose`, `openssl`, `curl`, `jq`.

- `tests/letsencrypt/cert_analyzer.sh` (sourced for the S2 verdicts).

Usage

```
bash deploy/letsencrypt/phase5_rotation.sh
KEEP_UP=1 bash deploy/letsencrypt/phase5_rotation.sh
CADDY_HTTPS_PORT=9443 CADDY_HTTP_PORT=9080 bash ...
PHASE5_NO_SURGERY=1 bash ...   # negative control: NO surgery =>
        NO renewal (guard RED)
```

Exit codes

Code	Meaning
0	PASS — rotation S1→S2 (new serial) + 0 dropped across the swap + analyzer verifies S2
1	FAIL — no rotation, dropped requests during the swap, or analyzer failed
2	OPERATOR-BLOCKED / precondition unmet (image not built, jq/podman-compose absent)

What it verifies

- **Rotation:** the served serial changes (S1 → S2) and S2's notBefore > S1's.
- **Zero-downtime swap:** a continuous `curl … /health` probe across the renewal records **0** failures (the probe runs *after* the restart, so it measures the swap, not the restart).
- **cert-analyzer over S2:** `cert_not_expired` + `cert_san_matches` + `cert_chain_roots_in` (S2 chains to **this run's** Pebble CA — proves a genuine new issuance).

Internal behaviour

1. Issue S1 via `phase3_hermetic_issue.sh` (admin API on, `KEEP_UP`).
2. Rewrite the cached ARI window + `_selectedTime` to the past (skipped iff `PHASE5_NO_SURGERY=1`).
3. Restart Caddy (re-loads the past window from storage).
4. Wait for Caddy to serve again, then probe availability + poll for the new serial.
5. Verdict: rotation ∧ 0 dropped ∧ analyzer verifies S2 → PASS. Evidence under `qa-results/letsencrypt/phase5_rotation/<run-id>/`.

Related

- `deploy/letsencrypt/phase3_hermetic_issue.sh` — the issuance proof this builds on.
- `tests/letsencrypt/phase5_rotation_guard.sh` — the §11.4.135 standing guard (RED_MODE).
- `docs/research/caddy_2110_ari_refetch_20260701/` + `pebble_set_renewal_info_20260701/` — the mechanism research.

Last verified

2026-07-01 — clean run: S1 → S2 (distinct serials, S2 notBefore later), `swap ok=6 fails=0`, cert-analyzer PASS (chain to per-run Pebble CA). Evidence: `qa-results/letsencrypt/phase5_rotation/20260701T113915Z/`.